Intro to Classification Problems

মেশিন লার্নিং এর সমস্যার ধরন

মেশিন লার্নিং-এ প্রধানত দুই ধরনের সমস্যা নিয়ে আলোচনা করা হয়:

১. রিগ্রেশন (Regression)

কী ধরণের সমস্যা সমাধান করে: যেখানে আউটপুট বা প্রেডিকশন একটি কন্টিনিউয়াস ভ্যালু **(Continuous Value)** হয়।
কন্টিনিউয়াস ভ্যালু মানে: যে মান যেকোনো সংখ্যা হতে পারে, যেমন ১০০, ১০০.৫, ২৫০০০ ইত্যাদি। এটা কোনো শ্রেণিতে সীমাবদ্ধ নয়।
উদাহরণ:একটি বাড়ির দাম (House Price)
একজন ব্যক্তির বেতন (Salary)
পরীক্ষার প্রাপ্ত নম্বর (Marks)
মনে রাখার সহজ উপায়: রিগ্রেশন = “কতটা?” (How much?) বা “কী মান?” (What value?) - উত্তরটা একটি সংখ্যা।

২. ক্লাসিফিকেশন (Classification)

কী ধরণের সমস্যা সমাধান করে: যেখানে আউটপুট বা প্রেডিকশন কয়েকটি নির্দিষ্ট ক্যাটাগরি বা শ্রেণির **(Category/Class)** মধ্যে একটি হয়।
উদাহরণ ও প্রকারভেদ:

ক. বাইনারি ক্লাসিফিকেশন (Binary Classification)

সংজ্ঞা: যখন শুধু দুইটি ক্যাটাগরি থাকে। সাধারণত হ্যাঁ/না, সত্য/মিথ্যা, ০/১ টাইপের।
উদাহরণ:ইমেইলটি স্প্যাম নাকি নট স্প্যাম?
ছাত্রটি পাস করবে নাকি ফেল করবে?
টাইটানিক জাহাজের যাত্রীটি বাঁচবেন নাকি মারা যাবেন?
মনে রাখবেন: এখানে দুটি ক্লাস। যেমন: পাস (১) অথবা ফেল (০)।

খ. মাল্টিক্লাস ক্লাসিফিকেশন (Multiclass Classification)

সংজ্ঞা: যখন দুইটির বেশি ক্যাটাগরি থাকে।
উদাহরণ:একজন ভোটার বিএনপি, আওয়ামী লীগ, জামায়াত নাকি অন্য কাউকে ভোট দেবেন? (এখানে ৪টি ক্লাস)।
একটি ছবিতে কোন প্রাণীটি আছে: কুকুর, বিড়াল, নাকি পাখি?
মনে রাখবেন: একাধিক ক্লাসের মধ্যে যেকোনো একটি প্রেডিক্ট করা।

লজিস্টিক রিগ্রেশন (Logistic Regression)

নাম নিয়ে প্রথমে একটি বড় ভুল ধারণা দূর করুন

নামে ‘রিগ্রেশন’ থাকলেও এটি কিন্তু রিগ্রেশন প্রবলেম সমাধান করে না।
এটি আসলে একটি ক্লাসিফিকেশন অ্যালগরিদম।

লজিস্টিক রিগ্রেশন কী সমস্যা সমাধান করে?

এটি মূলত বাইনারি ক্লাসিফিকেশন (দুইটি ক্লাস) সমস্যা সমাধান করে।
অর্থাৎ, কোনো কিছু হওয়ার সম্ভাবনা বনাম না হওয়ার সম্ভাবনা বের করে।

কেন নাম লজিস্টিক রিগ্রেশন?

লিনিয়ার রিগ্রেশন থেকে আমরা একটি কন্টিনিউয়াস সংখ্যা পাই (যেমন ৫০, -১০, ২০০ ইত্যাদি)।

লজিস্টিক রিগ্রেশন সেই সংখ্যাটিকে একটি বিশেষ ফাংশনের (লজিস্টিক ফাংশন বা সিগময়েড ফাংশন) মাধ্যমে ০ থেকে ১ এর মধ্যে একটি সম্ভাবনায় **(Probability)** রূপান্তর করে।
তারপর সেই সম্ভাবনার ভিত্তিতে সিদ্ধান্ত নেওয়া হয় (যেমন: ০.৫ এর বেশি হলে 'হ্যাঁ', কম হলে 'না')।

Key concepts of Logistic Regression

লজিস্টিক রিগ্রেশন কীভাবে কাজ করে?

১. লজিস্টিক রিগ্রেশন একটি সুপারভাইজড লার্নিং অ্যালগরিদম

সুপারভাইজড লার্নিং মানে: মডেলকে ট্রেনিং করার সময় টার্গেট কলাম **(True Value / Ground Truth)** দিতে হয়।
উদাহরণ**:** টাইটানিক ডেটাসেটে প্রতিটি যাত্রীর জন্য সে বাঁচবে কি না (১ বা ০) আগেই দেওয়া থাকে। মডেল এই সঠিক উত্তর দেখে শেখে।

২. মডেল কীভাবে শেখে? (লার্নিং সাইকেল)

মডেলের কিছু প্যারামিটার থাকে (যেমন ঢাল ও ইন্টারসেপ্ট)।
মডেল ডেটা পয়েন্ট দেখে একটি প্রেডিকশন করে।
প্রেডিকশনের সাথে একচুয়াল ভ্যালু (সঠিক উত্তর) তুলনা করে।
ভুল হলে কস্ট **(Cost)** বা পেনাল্টি গণনা করে।
পেনাল্টি কমাতে প্যারামিটার আপডেট করে।
এভাবে বারবার চেষ্টা করে কস্ট মিনিমাইজ করে।

💡 মনে রাখবেন**:** কস্ট ফাংশন = মডেলের ভুলের পরিমাণ। লার্নিং মানেই এই কস্ট কমানো।

৩. লজিস্টিক রিগ্রেশন কী আউটপুট দেয়?

এটা ডিরেক্টলি ০ বা ১ বলে না। বরং বলে:
"এই ব্যক্তি বাঁচার সম্ভাবনা **(Probability)** কত?"

সম্ভাবনা সবসময় ০ থেকে ১ এর মধ্যে থাকে।
০.৫২ মানে → ৫২% সম্ভাবনা বাঁচবে, ৪৮% সম্ভাবনা বাঁচবে না।
০.৯৫ মানে → ৯৫% সম্ভাবনা বাঁচবে, ৫% সম্ভাবনা বাঁচবে না।

৪. থ্রেশহোল্ড (Threshold) কী?

থ্রেশহোল্ড **=** সিদ্ধান্তের সীমানা। সাধারণত ধরা হয় ০.৫।
নিয়ম:Probability ≥ ০.৫ → ক্লাস = ১ (বাঁচবে)
Probability < ০.৫ → ক্লাস = ০ (বাঁচবে না)
উদাহরণ**:** মডেল বলল ০.৪৮ (৪৮%)। যেহেতু ০.৫-এর নিচে, সিদ্ধান্ত নেবেন: বাঁচবে না।

৫. কেন শুধু ০/১ না বলে সম্ভাবনা দরকার?

কারণ কনফিডেন্স **(**আত্মবিশ্বাস**)** পরিমাপ করতে হয়।

উদাহরণ: দুই ধরনের ভুল

পরিস্থিতি	মডেলের প্রেডিকশন (Probability)	একচুয়াল ভ্যালু	কনফিডেন্স লেভেল	পেনাল্টি (কস্ট)
কেস ১	০.৯৫ (৯৫% নিশ্চিত বাঁচবে)	০ (বাঁচেনি)	**অনেক বেশি**	**অনেক বেশি** (ভুলের দাম বেশি)
কেস ২	০.৫২ (৫২% নিশ্চিত বাঁচবে)	০ (বাঁচেনি)	**সামান্য বেশি**	**কম** (তেমন আত্মবিশ্বাস ছিল না)

🔥 মূল কথা: বেশি আত্মবিশ্বাসের সাথে ভুল করলে পেনাল্টি বেশি দেয়া হয়। এতে মডেল শেখে যে ভুলের মূল্য অনেক।

৬. লিনিয়ার রিগ্রেশন দিয়ে ক্লাসিফিকেশন করলে সমস্যা কী?

লিনিয়ার রিগ্রেশন সরাসরি ০/১ প্রেডিক্ট করলে:

- কস্ট ফাংশন কনভেক্স **(Convex)** থাকবে না।
- গ্রেডিয়েন্ট ডিসেন্ট কাজ করবে না (স্থানীয় মিনিমামে আটকে যেতে পারে)।
- ভুলের পরিমাণ সঠিকভাবে বোঝা যাবে না।

লজিস্টিক রিগ্রেশন সম্ভাবনার মাধ্যমে কনফিডেন্স মেপে এই সমস্যার সমাধান করে।

Limitation of Linear regression for Classification Problem

লিনিয়ার রিগ্রেশন দিয়ে ক্লাসিফিকেশন করলে কেন কাজ করে না?

১. ডেটাসেট ও সমস্যা বুঝুন

- ফিচার **(Feature):** টিকেটের দাম (Ticket Price) – একটাই ফিচার।
- টার্গেট **(Target):** বাঁচবে কি না (Categorical Column) – মাত্র দুইটি ক্লাস:
  - ০ **(Not Survived):** বাঁচেনি।
  - ১ **(Survived):** বেঁচে গেছে।

এখন এই ডেটা প্লট করলে X-অক্ষে টিকেটের দাম, Y-অক্ষে ০/১ হবে। Y-অক্ষের মান হয় ০ না হয় ১ – এর মাঝখানে কোনো মান নেই।

২. লিনিয়ার রিগ্রেশন দিয়ে বেস্ট ফিট লাইন আঁকা

লিনিয়ার রিগ্রেশন চেষ্টা করবে একটি সরলরেখা **(Best Fit Line)** আঁকতে, যাতে সব পয়েন্ট থেকে লাইনের দূরত্ব সর্বনিম্ন হয়।

- এই লাইন Y-অক্ষের মান ০ থেকে ১ এর বাইরেও যেতে পারে (যেমন ১.৫ বা -০.৩)।
- এরপর আমরা থ্রেশহোল্ড **(Threshold)** ধরি, সাধারণত ০.৫।
- ০.৫ এর উপরে → ক্লাস ১ (বাঁচবে)
- ০.৫ এর নিচে → ক্লাস ০ (বাঁচবে না)

এভাবে একটি ডিসিশন বাউন্ডারি তৈরি হয়, যা লাইনটিকে দুইভাগে ভাগ করে।

৩. প্রথম সমস্যা: ডিসিশন বাউন্ডারি অস্থির (Unstable)

লিনিয়ার রিগ্রেশনের লাইন সব ডেটা পয়েন্টের গড় দূরত্ব মিনিমাইজ করতে চায়।

- যদি নতুন আউটলায়ার (অন্য পয়েন্ট) আসে, পুরো লাইন সরে যায়।

যার ফলে ডিসিশন বাউন্ডারির অবস্থান বদলে যায়।
আগে যে পয়েন্টটি সঠিক ক্লাসে পড়ছিল, পরে ভুল ক্লাসে পড়তে পারে।
মূল কথা: আউটলায়ার থাকলে লাইন অতিরিক্ত সংবেদনশীল (sensitive) হয়ে পড়ে।

৪. দ্বিতীয় সমস্যা: আউটপুট সম্ভাবনার মধ্যে নেই

লিনিয়ার রিগ্রেশনের আউটপুট কন্টিনিউয়াস এবং এর রেঞ্জ:

$$(-\infty) \text{ to } (+\infty)$$

মানে আউটপুট -১০০, ০, ৫০০, যেকোনো সংখ্যা হতে পারে।
কিন্তু প্রোবাবিলিটি **(Probability)** সবসময় ০ থেকে ১ এর মধ্যে থাকে।
১.২ বা -০.৪ কোনো সম্ভাবনা হতে পারে না।

🔥 ইস্যু: আমরা যখন মডেল ট্রেন করি, কস্ট ফাংশন ঠিকমতো কাজ করে না। কারণ কস্ট ফাংশন কনফিডেন্স অনুযায়ী পেনাল্টি দেয়, কিন্তু লিনিয়ার রিগ্রেশনের আউটপুট অসীম রেঞ্জের হওয়ায় সঠিক পেনাল্টি মাপা যায় না।

৫. সংক্ষেপে দুটি বড় সমস্যা

সমস্যা নং	সমস্যার বর্ণনা	প্রভাব
১	ডিসিশন বাউন্ডারি আউটলায়ারের কারণে ঘুরে যায়	প্রেডিকশন ভুল হয়, ক্লাস সীমানা ঠিক থাকে না
২	আউটপুট রেঞ্জ -∞ থেকে +∞, অথচ সম্ভাবনা লাগবে ০ থেকে ১	কস্ট ফাংশন সঠিক নয়, কনফিডেন্স মাপা যায় না

৬. কেন লজিস্টিক রিগ্রেশন ব্যবহার করব?

লজিস্টিক রিগ্রেশন এই দুটি সমস্যার সমাধান করে:

ডিসিশন বাউন্ডারি আউটলায়ার দ্বারা সহজে প্রভাবিত হয় না।
আউটপুট সিগময়েড ফাংশনের মাধ্যমে ০ থেকে ১ এর মধ্যে সীমাবদ্ধ থাকে, যা সঠিকভাবে সম্ভাবনা বোঝায়।
কনফিডেন্স অনুযায়ী পেনাল্টি দেয়া যায় (Log Loss).

Characteristics of sigmoid Function

সিগময়েড ফাংশন

১. সিগময়েড ফাংশন কেন দরকার?

আমরা আগে দেখেছি, লিনিয়ার রিগ্রেশনের দুটি বড় সমস্যা ছিল:

কস্ট ফাংশনের সমস্যা: লিনিয়ার রিগ্রেশনের কস্ট ফাংশন ক্লাসিফিকেশনের জন্য ঠিকমতো কাজ করে না।
প্রবাবিলিটির সমস্যা: আমরা চাই একটি প্রবাবিলিটি **(Probability)** বা কনফিডেন্স – কোনো ঘটনা ঘটবে কি না (যেমন বাঁচবে কি না, স্প্যাম কি না)। কিন্তু লিনিয়ার রিগ্রেশন আউটপুট দেয় -∞ থেকে +∞ পর্যন্ত, যা কখনো ০ থেকে ১ এর মধ্যে থাকে না।

✅ সমাধান: সিগময়েড ফাংশন। এই ফাংশন যেকোনো রিয়েল সংখ্যাকে (যেমন লিনিয়ার রিগ্রেশনের আউটপুট) ০ থেকে ১ এর মধ্যে রূপান্তর করে, যা একটি সুন্দর সম্ভাবনা হিসেবে ব্যাখ্যা করা যায়।

২. সিগময়েড ফাংশনের সূত্র

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

এখানে,

σ = সিগময়েড ফাংশনের আউটপুট (সর্বদা ০ থেকে ১ এর মধ্যে)
z = ইনপুট (যে কোনো বাস্তব সংখ্যা, যেমন লিনিয়ার রিগ্রেশনের আউটপুট)
e = অয়লার সংখ্যা (Euler's Number) ≈ **2.71828** (এটি একটি বিখ্যাত গাণিতিক ধ্রুবক)

৩. সিগময়েড ফাংশনের আচরণ (বিভিন্ন z-এর জন্য)

ক) যখন z = 0

$$\sigma(0) = \frac{1}{1 + e^{0}} = \frac{1}{1 + 1} = \frac{1}{2} = 0.5$$

➡️ আউটপুট **= 0.5** (এটাই থ্রেশহোল্ড)

খ) যখন z পজিটিভ (z > 0)

উদাহরণ: z = 2

$$\sigma(2) = \frac{1}{1 + e^{-2}} \approx \frac{1}{1 + 0.1353} \approx \frac{1}{1.1353} \approx 0.88$$

z যত বড় হবে, e^-z তত ছোট হবে।
তখন 1 + e^-z ≈ 1 এর কাছাকাছি হবে।
তাই σ(z) ≈ 1 এর কাছাকাছি হবে।

➡️ **z** পজিটিভ → আউটপুট **0.5** থেকে বড় এবং ১ এর কাছাকাছি যায়

গ) যখন z নেগেটিভ (z < 0)

উদাহরণ: z = -2

$$\sigma(-2) = \frac{1}{1 + e^{-(-2)}} = \frac{1}{1 + e^2} \approx \frac{1}{1 + 7.389} \approx \frac{1}{8.389} \approx 0.12$$

z নেগেটিভ হলে e^-z = e^positive বড় সংখ্যা হয়।
তাহলে 1 + e^-z অনেক বড় হয়।
তাই ভগ্নাংশটি ০ এর কাছাকাছি হয়।

➡️ **z** নেগেটিভ → আউটপুট **0.5** থেকে ছোট এবং ০ এর কাছাকাছি যায়

৪. সিগময়েড ফাংশনের গ্রাফ (S-আকৃতি)

z-এর মান (ইনপুট)	σ(z) (আউটপুট)
-∞ (মাইনাস ইনফিনিটি)	→ ০
-১০	খুব কাছাকাছি ০
-২	≈ ০.১২
-১	≈ ০.২৭
০	**০.৫** (থ্রেশহোল্ড)
১	≈ ০.৭৩
২	≈ ০.৮৮
১০	≈ ০.৯৯৯৯
+∞ (পজিটিভ ইনফিনিটি)	→ ১

গ্রাফের আকৃতি: 'S' আকারের (লজিস্টিক কার্ভ)

বাম দিকে (z নেগেটিভ) → মান ০ এর দিকে
মাঝখানে (z = ০) → মান ০.৫
ডান দিকে (z পজিটিভ) → মান ১ এর দিকে

৫. সিগময়েড ফাংশনের বৈশিষ্ট্য

বৈশিষ্ট্য	বিবরণ
আউটপুট রেঞ্জ	(০, ১) – কখনোই ঠিক ০ বা ১ নয়, কিন্তু অসীমের কাছে গেলে প্রায় ০ বা ১ হয়
মান যখন z=০	০.৫ (এটাকে থ্রেশহোল্ড ধরা হয়)
মান যখন z বড় পজিটিভ	প্রায় ১ (অর্থাৎ "হ্যাঁ" হওয়ার সম্ভাবনা অনেক বেশি)
মান যখন z বড় নেগেটিভ	প্রায় ০ (অর্থাৎ "না" হওয়ার সম্ভাবনা অনেক বেশি)
আকৃতি	স্মুথ S-আকৃতি (মসৃণ, ডিফারেন্সিয়েবল – যা গ্রেডিয়েন্ট ডিসেন্টের জন্য দরকার)

৬. প্রোগ্রামিং-এ সিগময়েড ফাংশন

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# z-এর বিভিন্ন মানের জন্য পরীক্ষা
z_values = [-10, -2, -1, 0, 1, 2, 10]
for z in z_values:
    print(f"sigmoid({z}) = {sigmoid(z):.4f}")
```

আউটপুট (আনুমানিক):

```
sigmoid(-10) = 0.0000
sigmoid(-2) = 0.1192
sigmoid(-1) = 0.2689
sigmoid(0) = 0.5000
sigmoid(1) = 0.7311
sigmoid(2) = 0.8808
sigmoid(10) = 0.9999
```

Transition of Data

লিনিয়ার রিগ্রেশন → লজিস্টিক রিগ্রেশন: কীভাবে কাজ করে?

১. লিনিয়ার রিগ্রেশনের ফাংশন (Recall)

লিনিয়ার রিগ্রেশনের আউটপুট ছিল একটি কন্টিনিউয়াস ভ্যালু (যেকোনো সংখ্যা, যেমন -১০, ০, ৫০, ১০০০...):

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

এখানে,

w = ওয়েট (Weight)
x = ফিচার (Feature)
b = বায়াস (Bias)

এই z -কে আমরা র **(Raw Output)** বলি। এটি কোনো সীমার মধ্যে আবদ্ধ না।

২. লজিস্টিক রিগ্রেশনে রূপান্তর

লজিস্টিক রিগ্রেশনে আমরা এই একই \(z \)-কে সিগময়েড ফাংশনে ইনপুট দিই:

$$f_{w,b}(x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

যেখানে,

$$z = w \cdot x + b = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

মূল কথা:
লিনিয়ার রিগ্রেশনের কাঠামো একই রাখি, শুধু শেষে সিগময়েড ফাংশন বসাই → আউটপুট হয়ে যায় ০ থেকে ১ এর মধ্যে (একটি সম্ভাবনা)।

৩. সিগময়েড আউটপুটকে কীভাবে ব্যাখ্যা করব?

সিগময়েড ফাংশনের আউটপুট হলো পজিটিভ ক্লাস **(১)** হওয়ার সম্ভাবনা।

উদাহরণ (টাইটানিক ডেটাসেট):

ফিচার: টিকেটের মূল্য, বয়স, কেবিন নম্বর ইত্যাদি।
টার্গেট: ১ = বাঁচবে, ০ = বাঁচবে না।

মডেলটি একটি যাত্রীর জন্য \(\sigma(z) = 0.82 \) দিলে:

বাঁচার সম্ভাবনা = ৮২% (P(সারভাইভ) = 0.82)
না বাঁচার সম্ভাবনা = ১ – ০.৮২ = ০.১৮ = ১৮% (P(নট সারভাইভ) = 0.18)

💡 সাধারণ নিয়ম: সম্ভাবনা ≥ ০.৫ হলে প্রেডিক্ট করি ক্লাস ১, নইলে ক্লাস ০।

৪. কনফিডেন্স (Confidence) কী এবং কেন দরকার?

এখানে ০.৮২ শুধু একটি সম্ভাবনা নয়, এটি মডেলের আত্মবিশ্বাসের মাত্রা-ও বোঝায়।

উচ্চ কনফিডেন্স + সঠিক উত্তর → সামান্য পেনাল্টি।
উচ্চ কনফিডেন্স + ভুল উত্তর → অনেক বড় পেনাল্টি। (উদাহরণ: মডেল ৯৫% নিশ্চিত বাঁচবে বলে, কিন্তু যাত্রীটি মারা যায় → বড় কস্ট যোগ হবে)

লিনিয়ার রিগ্রেশন এই "কনফিডেন্স-ভিত্তিক পেনাল্টি" দিতে পারে না, কারণ তার আউটপুট -∞ থেকে +∞। সিগময়েডের মাধ্যমে আমরা সঠিকভাবে কনফিডেন্স পরিমাপ করতে পারি।

Cost function of Logistic Regression

লজিস্টিক রিগ্রেশনের কস্ট ফাংশন (Log Loss)

১. কেন লিনিয়ার রিগ্রেশনের কস্ট ফাংশন ব্যবহার করব না?

লিনিয়ার রিগ্রেশনে কস্ট ফাংশন ছিল **Mean Squared Error (MSE)**:

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - \hat{y}^{(i)})^2$$

লজিস্টিক রিগ্রেশনে এটি কাজ করে না, কারণ:

আউটপুট ও টার্গেট ভিন্ন ধরনের:
লিনিয়ার রিগ্রেশন: প্রেডিকশন কন্টিনিউয়াস, টার্গেট কন্টিনিউয়াস।
লজিস্টিক রিগ্রেশন: প্রেডিকশন প্রবাবিলিটি (০ থেকে ১), টার্গেট ০ বা ১। সরাসরি বিয়োগ করলে অর্থপূর্ণ পেনাল্টি পাওয়া যায় না।
কস্ট ফাংশন কনভেক্স থাকে না:
MSE ব্যবহার করলে কস্ট ফাংশন নন-কনভেক্স হয় (অর্থাৎ অনেক স্থানীয় মিনিমা থাকে)।
ফলে গ্রেডিয়েন্ট ডিসেন্ট গ্লোবাল মিনিমামে পৌঁছাতে পারে না।

✅ সমাধান: লজিস্টিক রিগ্রেশনের জন্য আলাদা কস্ট ফাংশন দরকার – **Log Loss** (Binary Cross-Entropy)।

২. Log Loss ফাংশনের স্বজ্ঞাত ব্যাখ্যা (Intuition)

ক) যখন একচুয়াল টার্গেট y = 1 (যেমন: বাঁচবে)

পরিস্থিতি	মডেলের প্রেডিকশন ŷ	কনফিডেন্স	কস্ট
সঠিক ও আত্মবিশ্বাসী	১.০	১০০%	০
সঠিক, কিন্তু কম আত্মবিশ্বাসী	০.৮	৮০%	কম (যেমন ০.২২)
ভুল ও আত্মবিশ্বাসী	০.০	০%	**অনেক বেশি** (যেমন ইনফিনিটির কাছাকাছি)

📌 আমরা চাই: ভবিষ্যদ্বাণী যত ভুল ও আত্মবিশ্বাসী, পেনাল্টি তত বেশি।
গাণিতিক সূত্র **y=1** হলে**):**

$$\text{Cost} = -\log(\hat{y})$$

- ŷ = 1 → -log(1) = 0 (কোনো কস্ট নেই)
- ŷ = 0.9 → -log(0.9) ~0.105 (সামান্য কস্ট)
- ŷ = 0.5 → -log(0.5) ~0.693 (মাঝারি কস্ট)
- ŷ → 0 → -log(ŷ) → ∞ (অসীম কস্ট)

খ) যখন একচুয়াল টার্গেট y = 0 (যেমন: বাঁচবে না)
এখানে ভুল হচ্ছে মডেল যদি বেশি আত্মবিশ্বাসের সাথে বাঁচবে বলে (যদিও বাঁচেনি)।
গাণিতিক সূত্র **y=0** হলে**):**

$$\text{Cost} = -\log(1 - \hat{y})$$

- ŷ = 0 → -log(1) = 0
- ŷ = 0.2 → -log(0.8) ~0.223 (কম কস্ট)
- ŷ = 0.8 → -log(0.2) ~1.609 (বেশি কস্ট)
- ŷ → 1 → -log(0) → ∞ (অসীম কস্ট)

📌 মূল বিষয়: মডেল যত বেশি আত্মবিশ্বাসের সাথে ভুল করবে, কস্ট তত বেশি হবে।

৩. একটি ডেটা পয়েন্টের জন্য কস্ট (লস ফাংশন)
উপরের দুই কেস এক সূত্রে লেখা যায়:

$$\mathcal{L}(\hat{y}, y) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

যাচাই:

- $y = 1$ হলে: $\mathcal{L} = -\log(\hat{y})$ (দ্বিতীয় টার্ম ০)
- $y = 0$ হলে: $\mathcal{L} = -\log(1 - \hat{y})$ (প্রথম টার্ম ০)

একে বলা হয় **Log Loss** বা **Binary Cross-Entropy Loss**।

৪. পুরো ডেটাসেটের জন্য কস্ট ফাংশন (Cost Function)

সব m টি ট্রেনিং উদাহরণের গড় লস নিই:

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

$$J(w,b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{y}^{(i)}\right) + (1-y^{(i)})\log\left(1-\hat{y}^{(i)}\right)\right]$$

এখানে:

- m = ডেটা পয়েন্টের সংখ্যা
- $y^{(i)}$ = i-তম উদাহরণের প্রকৃত টার্গেট (০ বা ১)
- $\hat{y}^{(i)} = \sigma(w \cdot x^{(i)} + b)$ = মডেলের প্রেডিক্টেড সম্ভাবনা

৫. নামকরণ ও বিকল্প নাম

নাম	ব্যাখ্যা
Log Loss	লস (এরর) এর লগারিদম নেওয়া হয় বলে
Binary Cross-Entropy	বাইনারি ক্লাসিফিকেশনের জন্য ক্রস-এন্ট্রপি (বিশৃঙ্খলা মাপার পদ্ধতি)

ক্রস-এন্ট্রপি আসলে দুটি ডিস্ট্রিবিউশনের (প্রেডিক্টেড vs একচুয়াল) মধ্যে পার্থক্য মাপে।

Optimizing the cost function

লজিস্টিক রিগ্রেশনের কস্ট ফাংশন অপটিমাইজেশন (গ্রেডিয়েন্ট ডিসেন্ট)

১. আমরা কোথায় আছি?

> আগে শিখেছি: লজিস্টিক রিগ্রেশনের কস্ট ফাংশন **(Log Loss)**।
> লক্ষ্য: এই কস্ট ফাংশন মিনিমাইজ করা।
> কস্ট ফাংশন মিনিমাইজ মানেই: মডেলের ভুল কমানো → ভালো প্রেডিকশন।

২. গ্রেডিয়েন্ট ডিসেন্টের সাধারণ সূত্র (Recall)

লিনিয়ার রিগ্রেশনের মতো এখানেও গ্রেডিয়েন্ট ডিসেন্ট ব্যবহার করি প্যারামিটার আপডেট করার জন্য।

ওয়েট **(w)** আপডেটের নিয়ম:

$$w_j := w_j - \alpha \frac{\partial J(w,b)}{\partial w_j}$$

বায়াস **(b)** আপডেটের নিয়ম:

$$b := b - \alpha \frac{\partial J(w,b)}{\partial b}$$

এখানে,

α = লার্নিং রেট (শেখার হার)
J(w,b) = কস্ট ফাংশন

৩. লজিস্টিক রিগ্রেশনের জন্য ডেরিভেটিভ (Derivative)

ভালো খবর: লজিস্টিক রিগ্রেশনের ডেরিভেটিভ লিনিয়ার রিগ্রেশনের মতোই সুন্দর ও সহজ!

আমরা পাই:

$$\frac{\partial J(w,b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial J(w,b)}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})$$

এখানে,

- $\hat{y}^{(i)} = \sigma(w \cdot x^{(i)} + b)$ = প্রেডিক্টেড সম্ভাবনা (০ থেকে ১)
- $y^{(i)}$ = প্রকৃত টার্গেট (০ বা ১)
- $x_j^{(i)}$ = i-তম উদাহরণের j-তম ফিচার

🔥 মনে রাখার বিষয়:

সূত্রটি দেখতে লিনিয়ার রিগ্রেশনের গ্রেডিয়েন্টের মতোই!

শুধু পার্থক্য: এখানে সিগময়েড ফাংশনের আউটপুট (প্রবাবিলিটি), আগে ছিল শুধু লিনিয়ার আউটপুট।

৪. লিনিয়ার রিগ্রেশন vs লজিস্টিক রিগ্রেশন (গ্রেডিয়েন্ট)

বিষয়	লিনিয়ার রিগ্রেশন	লজিস্টিক রিগ্রেশন
মডেল আউটপুট	$\hat{y} = w \cdot x + b$	$\hat{y} = \sigma(w \cdot x + b)$
কস্ট ফাংশন	MSE	Log Loss
গ্রেডিয়েন্ট w-এর জন্য	$\frac{1}{m}\sum(\hat{y} - y)x_j$	$\frac{1}{m}\sum(\hat{y} - y)x_j$ (একই ফরম!)
গ্রেডিয়েন্ট b-এর জন্য	$\frac{1}{m}\sum(\hat{y} - y)$	$\frac{1}{m}\sum(\hat{y} - y)$ (একই ফরম!)

✅ বিশেষ টেকওয়ে:

গ্রেডিয়েন্টের ফর্মুলা একই! শুধু $\hat{y}$ -এর সংজ্ঞা আলাদা।

তাই লিনিয়ার রিগ্রেশনের কোড থেকে সামান্য পরিবর্তন করেই লজিস্টিক রিগ্রেশন বানানো যায়।

৫. গ্রেডিয়েন্ট ডিসেন্ট অ্যালগরিদম (ধাপে ধাপে)

ইনিশিয়ালাইজ w এবং b (যেমন ০ বা ছোট র‍্যান্ডম মান)

লুপ যতক্ষণ না কনভার্জ হয়:

- প্রতিটি ডেটা পয়েন্টের জন্য $\hat{y}^{(i)} = \sigma(w \cdot x^{(i)} + b)$ হিসাব কর
- হিসাব কর $\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum(\hat{y}^{(i)} - y^{(i)})x_j^{(i)}$
- হিসাব কর $\frac{\partial J}{\partial b} = \frac{1}{m}\sum(\hat{y}^{(i)} - y^{(i)})$
- আপডেট কর $w_j := w_j - \alpha\frac{\partial J}{\partial w_j}$
- আপডেট কর $b := b - \alpha\frac{\partial J}{\partial b}$

রিপিট যতক্ষণ কস্ট ফাংশন কমতে থাকে (বা নির্দিষ্ট সংখ্যক বার)

৬. কনভার্জেন্স (Convergence) বলতে কী বোঝায়?

কনভার্জ মানে: কস্ট ফাংশনের মান আর উল্লেখযোগ্য হারে কমছে না।

গ্রাফিক্যালি: কস্ট ফাংশনের গ্লোবাল মিনিমামে পৌঁছানো।

লজিস্টিক রিগ্রেশনের কস্ট ফাংশন কনভেক্স → গ্লোবাল মিনিমাম নিশ্চিত (স্থানীয় মিনিমার সমস্যা নেই)।

৭. ইমপ্লিমেন্টেশনের ব্যবহারিক দিক (কোড না করলেও জানা দরকার)

আমরা স্ক্র্যাচ থেকে ইমপ্লিমেন্ট করব না।
বাস্তবে **scikit-learn** লাইব্রেরি ব্যবহার করব।
scikit-learn-এর LogisticRegression ক্লাস ভিতরে এই গণিতগুলোই করে।

তবুও পেছনের গণিত জানা দরকার, কারণ:

হাইপারপ্যারামিটার টিউন করতে সাহায্য করে।
সমস্যা হলে ডিবাগ করতে পারবেন।
কাস্টম মডেল বানাতে চাইলে কাজে লাগে।

Logistic Regresion from Scratch

স্ক্র্যাচ থেকে লজিস্টিক রিগ্রেশন ইমপ্লিমেন্টেশন

১. লিনিয়ার রিগ্রেশনের কোডের সাথে কী কী চেঞ্জ হবে?

লিনিয়ার রিগ্রেশনের মতোই সবকিছু, শুধু প্রেডিকশন ফাংশন ও কস্ট ফাংশন পরিবর্তন হয়।

যে অংশ	লিনিয়ার রিগ্রেশনে	লজিস্টিক রিগ্রেশনে চেঞ্জ
প্রেডিকশন	`z = np.dot(w,X[i])+b`	`p = sigmoid(z)`
কস্ট ফাংশন	MSE	Log Loss (দুটি টার্মের যোগফল)
গ্রেডিয়েন্টের এরর	`z - y[i]`	`p - y[i]` (একই ফরম)
আউটপুট	যেকোনো সংখ্যা	০ থেকে ১ (প্রবাবিলিটি) + থ্রেশহোল্ড

২. সিগময়েড ফাংশন তৈরি

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

এটি যেকোনো সংখ্যা z কে ০ থেকে ১ এর মধ্যে নিয়ে আসে।

৩. প্রেডিকশন ফাংশন (Probability বের করা)

লিনিয়ার রিগ্রেশনে ছিল:

```
z = np.dot(w, X[i]) + b
```

```
prediction = z    # সরাসরি সংখ্যা
```

লজিস্টিক রিগ্রেশনে:

```
z = np.dot(w, X[i]) + b
probability = sigmoid(z)    # ০ থেকে ১ এর মান
```

পুরো প্রেডিকশন ফাংশন (একবারে সব ডেটার জন্য):

```
def predict_probability(X, w, b):
    z = np.dot(X, w) + b
    return sigmoid(z)
```

বাইনারি ক্লাস (০/১) পেতে থ্রেশহোল্ড ০.৫ ব্যবহার করি:

```
def predict(X, w, b, threshold=0.5):
    prob = predict_probability(X, w, b)
    return prob >= threshold   # True/False রিটার্ন করবে
```

৪. কস্ট ফাংশন (Log Loss) ইমপ্লিমেন্টেশন

লিনিয়ার রিগ্রেশনের MSE-র জায়গায় Log Loss বসে:

$$J(w,b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{y}^{(i)}\right) + (1-y^{(i)})\log\left(1-\hat{y}^{(i)}\right)\right]$$

কোড (লুপ ব্যবহার করে, বুঝতে সহজ):

```
def compute_cost(X, y, w, b):
    m = X.shape[0]
    total_cost = 0
    for i in range(m):
        z = np.dot(w, X[i]) + b
        p = sigmoid(z)   # প্রেডিক্টেড প্রোবাবিলিটি
        total_cost += y[i] * np.log(p) + (1 - y[i]) * np.log(1 - p)
    cost = -total_cost / m
    return cost
```

৫. গ্রেডিয়েন্ট ডিসেন্ট (প্যারামিটার আপডেট)

গ্রেডিয়েন্টের ফর্মুলা লিনিয়ার রিগ্রেশনের মতোই, শুধু প্রেডিকশন \(\hat{y}\) এখন সিগময়েড আউটপুট:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})$$

গ্রেডিয়েন্ট গণনা ফাংশন:

```
def compute_gradients(X, y, w, b):
    m = X.shape[0]
    dw = np.zeros_like(w)
    db = 0
    for i in range(m):
        z = np.dot(w, X[i]) + b
        p = sigmoid(z)
        error = p - y[i]
        dw += error * X[i]
         db += error
     dw /= m
     db /= m
     return dw, db
```

গ্রেডিয়েন্ট ডিসেন্ট (ট্রেনিং লুপ):

```
def gradient_descent(X, y, w_init, b_init, alpha, iterations):
    w = w_init.copy()
    b = b_init
    for _ in range(iterations):
        dw, db = compute_gradients(X, y, w, b)
        w -= alpha * dw
        b -= alpha * db
    return w, b
```

৬. সম্পূর্ণ ইমপ্লিমেন্টেশন সংক্ষেপে

```
# ১. ডেটা তৈরি (উদাহরণ)
X = np.array([[1, 2], [2, 3], [3, 4], [4, 5]])
y = np.array([0, 0, 1, 1])
```

```
4
5# ২. ইনিশিয়ালাইজেশন
6w_init = np.array([0.1, 0.1])
7b_init = 0.0
8alpha = 0.01
9iterations = 1000
10
11# ৩. ট্রেনিং
12w_final, b_final = gradient_descent(X, y, w_init, b_init, alpha,
iterations)
13
14# ৪. প্রেডিকশন
15predictions = predict(X, w_final, b_final)
16print(predictions)   # [False False  True  True]


17
18# (যদি ০/১ চান)
19print(predictions.astype(int))   # [0 0 1 1]
```

৭. লিনিয়ার রিগ্রেশনের কোড থেকে কী চেঞ্জ করলাম?

যে অংশ	লিনিয়ার রিগ্রেশনে	লজিস্টিক রিগ্রেশনে চেঞ্জ
প্রেডিকশন	`z = np.dot(w,X[i])+b`	`p = sigmoid(z)`
কস্ট ফাংশন	MSE	Log Loss (দুটি টার্মের যোগফল)
গ্রেডিয়েন্টের এরর	`z - y[i]`	`p - y[i]` (একই ফরম)
আউটপুট	যেকোনো সংখ্যা	০ থেকে ১ (প্রবাবিলিটি) + থ্রেশহোল্ড

বাকি সবকিছু (গ্রেডিয়েন্ট ডিসেন্ট লুপ, আপডেট রুল) একই থাকে!

৮. গুরুত্বপূর্ণ টিপস

✅ এই কোড শুধু বোঝার জন্য। বাস্তবে আপনি **scikit-learn** ব্যবহার করবেন:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X, y)
predictions = model.predict(X)
```

✅ থ্রেশহোল্ড পরিবর্তন করতে চাইলে:

```
probabilities = model.predict_proba(X)[:, 1]   # ক্লাস ১ এর সম্ভাবনা
custom_pred = (probabilities >= 0.7).astype(int)   # থ্রেশহোল্ড ০.৭
```

✅ একুরেসি, প্রিসিশন, রিকল ইত্যাদি মেট্রিক দিয়ে মডেল মূল্যায়ন করতে হবে

Logistic Regression using Scikit

Logistic Regression কী এবং কেন?

আমরা এর আগে Linear Regression শিখেছি — যেখানে model একটি continuous value predict করে, যেমন বাড়ির দাম বা কোনো student-এর marks। কিন্তু real world-এ অনেক সময় আমাদের এমন প্রশ্নের উত্তর দরকার হয় যেখানে output শুধু দুটি category-র একটি — যেমন "এই passenger কি survive করেছে, নাকি করেনি?" এই ধরনের সমস্যা solve করতে আমরা ব্যবহার করি **Logistic Regression**।

Classification কী?

Machine Learning-এ যখন আমাদের goal হয় কোনো input-কে একটি নির্দিষ্ট **category** বা **class**-এ ফেলা, তখন সেই task-কে বলা হয় **Classification**। Classification-এ output একটি label — যেমন "yes" অথবা "no", "spam" অথবা "not spam", "sick" অথবা "healthy"।

```
Task                          |  Output Type       |  Algorithm
------------------------------|--------------------|--------------------
House price prediction        |  Continuous number |  Linear Regression
Passenger survival prediction |  yes / no          |  Logistic Regression
```

```
5Email spam detection           |  spam / not spam   |  Logistic
Regression
6Tumor diagnosis                |  malignant / not   |  Logistic
Regression
```

যখন output শুধুমাত্র দুটি **class**-এর একটি, তখন সেটাকে বলা হয় **Binary Classification**। Titanic survival prediction একটি Binary Classification সমস্যা — কারণ একজন passenger হয় survive করেছে ("yes"), নয়তো করেনি ("no")।

Logistic Regression কীভাবে কাজ করে?

নাম শুনে মনে হতে পারে এটি Regression-এর মতো কাজ করে। কিন্তু আসলে Logistic Regression একটি **Classification algorithm**। এটি প্রতিটি input-এর জন্য একটি **probability** calculate করে — অর্থাৎ কতটুকু সম্ভাবনা আছে যে output "yes" হবে।

```
1Input features                    |  Probability  |  Final Prediction
2-----------------------------------|---------------|-------------------
3Age=22, Sex=male, Pclass=third    |  0.31         |  no  (not survived)
4Age=28, Sex=female, Pclass=first  |  0.82         |  yes (survived)
```

Probability 0.5-এর বেশি হলে model predict করে "yes", কম হলে "no"। এই boundary-কে বলা হয় **decision threshold**।

Titanic Dataset পরিচিতি

এই module-এ আমরা **Titanic dataset** ব্যবহার করবো। Dataset-এ মোট **891** জন **passenger**-এর তথ্য আছে।

```
1Column       |  Description                  |  Example
2-------------|-------------------------------|----------------------
3PassengerId  |  Unique ID                    |  1, 2, 3
4Survived     |  Target column                |  "yes", "no"
5Pclass       |  Ticket class                 |  "first", "second",
"third"
6Name         |  Passenger name               |  Braund, Mr. Owen
7Sex          |  Gender                       |  "male", "female"
8Age          |  Age in years                 |  22.0, NaN
9SibSp        |  Siblings / Spouses onboard   |  1, 0
10Parch        |  Parents / Children onboard   |  0, 1
11Ticket       |  Ticket number                |  A/5 21171
12Fare         |  Ticket price                 |  7.25, 71.28
13Cabin        |  Cabin number                 |  C85, NaN
14Embarked     |  Port of embarkation          |  "S", "C", "Q"
```

লক্ষ্য করো — `Survived` column-এ "yes" এবং "no" string আছে, numeric 0/1 নয়। এই কারণে পরে আমাদের **encoding** করতে হবে। `Age` এবং `Cabin` column-এ অনেক missing value আছে — এগুলো handle করতে হবে **preprocessing** step-এ।

এই Module-এ আমরা কী করবো?

```
Step  |  Topic                    |  Details
------|---------------------------|----------------------------------
1     |  Feature Engineering      |  Create new columns
2     |  Train-Test Split         |  Divide data for training
3     |  Outlier Handling         |  Remove extreme values
4     |  Preprocessing Pipeline   |  Clean and scale data
5     |  Model Training           |  Fit Logistic Regression
6     |  Evaluation               |  Measure model performance
```

Summary

Logistic Regression একটি Classification algorithm যা input features দেখে probability calculate করে এবং সেই probability থেকে final class predict করে। Linear Regression continuous value predict করে, কিন্তু Logistic Regression category predict করে। Titanic dataset-এ আমাদের goal হলো একজন passenger survive করেছে কিনা সেটা predict করা।

Feature Engineering

Titanic dataset লোড করার পরে আমরা সরাসরি model train করতে পারি না। কারণ dataset-এ যে columns আছে সেগুলো সবসময় সেরা information দেয় না। **Feature Engineering** হলো সেই process যেখানে আমরা existing columns থেকে নতুন, আরও meaningful columns তৈরি করি যাতে model ভালোভাবে pattern শিখতে পারে।

Family_Size তৈরি করা

Dataset-এ দুটি আলাদা column আছে — `SibSp` (sibling ও spouse-এর সংখ্যা) এবং `Parch` (parent ও child-এর সংখ্যা)। এই দুটি column আলাদাভাবে model-কে বলে একজন passenger-এর কতজন relative আছে। কিন্তু model-এর জন্য আরও useful information হলো — সে মোট কতজন নিয়ে travel করছে।
এই কারণে আমরা একটি নতুন column তৈরি করি:

```
# SibSp + Parch = বাকি family members
# +1 = নিজেকে count করা
df['Family_Size'] = df['SibSp'] + df['Parch'] + 1
```

একজন passenger যদি একা travel করে, তার `SibSp = 0` এবং `Parch = 0` — তাহলে `Family_Size = 1`। যদি স্বামী-স্ত্রী একসাথে থাকে, `Family_Size = 2`। এই single column দিয়ে model অনেক সহজে বুঝতে পারে যে বড় family-র passenger-রা survive করেছে কিনা।

Deck তৈরি করা

`Cabin` column-এ প্রতিটি value-এর প্রথম character হলো **Deck** — অর্থাৎ জাহাজের কোন তলায় সেই passenger ছিল। যেমন `C85` মানে Deck C, `B78` মানে Deck B। Deck জানা গুরুত্বপূর্ণ কারণ upper deck-এর passenger-রা lifeboats-এর কাছাকাছি ছিল।

কিন্তু `Cabin` column-এ মাত্র 204টি non-null value আছে — বাকি 687টি missing। তাই আমরা আগে missing values fill করি, তারপর Deck extract করি:

```
# Missing cabin-গুলো "Missing" দিয়ে fill করা
df['Cabin'] = df['Cabin'].fillna("Missing")

# প্রথম character নিয়ে Deck column তৈরি
df['Deck'] = df['Cabin'].astype(str).str[0]
```

এই code চালানোর পর `Deck` column-এ যা পাওয়া যায়:

Deck	Count	Meaning
M	687	Missing (no cabin data)
C	59	Deck C
B	47	Deck B
D	33	Deck D
E	32	Deck E
A	15	Deck A
F	13	Deck F
G	4	Deck G
T	1	Deck T

"Missing" যাদের Cabin জানা নেই তাদের জন্য একটি আলাদা category হিসেবে কাজ করে। এতে missing data-র information-ও model শিখতে পারে।

X এবং y আলাদা করা

Feature Engineering শেষ হলে আমরা dataset-কে দুটি ভাগে ভাগ করি — **features (X)** এবং **target (y)**।

```
# X = model যা দেখে predict করবে
X = df.drop('Survived', axis=1)

# y = model যা predict করবে
y = df['Survived']
```

`X`-এ থাকবে `Survived` বাদে সব column। `y`-তে থাকবে শুধু `Survived` column যেখানে "yes" এবং "no" আছে। এই দুটি আলাদা রাখা জরুরি কারণ model কখনো নিজের answer দেখে শিখতে পারবে না।

Summary

Feature Engineering-এ আমরা `SibSp` এবং `Parch` মিলিয়ে `Family_Size` তৈরি করেছি এবং `Cabin` থেকে `Deck` extract করেছি। এই নতুন columns model-কে আরও meaningful pattern শিখতে সাহায্য করে। এরপর `X` এবং `y` আলাদা করে data splitting-এর জন্য ready করা হয়েছে।

Train-Test Split: random_state ও stratify

Feature Engineering শেষে আমাদের কাছে `X` এবং `y` ready আছে। এখন model train করার আগে একটি গুরুত্বপূর্ণ কাজ করতে হবে — data-কে দুটি ভাগে ভাগ করা। একটি ভাগ দিয়ে model শিখবে, আরেকটি ভাগ দিয়ে আমরা দেখবো model কতটা ভালো কাজ করছে।

Train এবং Test Set কেন দরকার?

ধরো তুমি একটি exam-এর আগে practice করছো। যদি তুমি শুধু সেই questions-ই practice করো যেগুলো actual exam-এ আসবে, তাহলে তোমার score ভালো হবে ঠিকই — কিন্তু তুমি আসলে কতটুকু শিখেছো সেটা বোঝা যাবে না। Model-এর ক্ষেত্রেও একই কথা প্রযোজ্য।

```
Train Set  |  Model এই data দেখে শেখে
Test Set   |  Model এই data আগে দেখেনি — এখানে evaluate করা হয়
```

যদি model train এবং test উভয় ক্ষেত্রে একই data ব্যবহার করা হয়, তাহলে model শুধু সেই data মুখস্থ করে ফেলবে — নতুন data-তে ভালো কাজ করবে না। এই সমস্যাকে বলা হয় **overfitting**।

train_test_split ব্যবহার

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

এই একটি line-এ চারটি parameter কাজ করছে। প্রতিটি parameter-এর আলাদা গুরুত্ব আছে।

test_size=0.2 মানে কী?

`test_size=0.2` মানে মোট data-এর **20% test set-**এ যাবে, বাকি **80% train set-**এ থাকবে।

```
Total data   |  891 rows
-------------|-----------------
Train set    |  713 rows (80%)
Test set     |  178 rows (20%)
```

80/20 split ML-এ একটি commonly used অনুপাত। Train set যত বড় হবে, model তত বেশি data থেকে শিখতে পারবে। Test set যত বড় হবে, evaluation তত reliable হবে।

random_state=42 মানে কী?

`train_test_split` data-কে randomly ভাগ করে। কিন্তু প্রতিবার run করলে যদি আলাদা আলাদা ভাগ হয়, তাহলে প্রতিবার model-এর result আলাদা হবে — এটা reproducible নয়।

`random_state=42` দিলে প্রতিবার একই ভাবে data ভাগ হবে। 42 সংখ্যাটি বিশেষ কিছু নয় — যেকোনো integer দেওয়া যায়। তবে team-এর সবাই একই `random_state` ব্যবহার করলে সবার কাছে একই result আসবে।

stratify=y মানে কী?

Titanic dataset-এ সবাই survive করেনি — কিছু "yes", কিছু "no"। যদি randomly split করা হয়, তাহলে হতে পারে train set-এ বেশিরভাগ "yes" চলে গেল এবং test set-এ বেশিরভাগ "no"। এই অসামঞ্জস্য model-এর learning-কে ক্ষতিগ্রস্ত করে।

`stratify=y` দিলে train এবং test উভয় set-এ **target class-**এর অনুপাত একই থাকে:

```
1              |  Total   |  Train set   |  Test set
2--------------|----------|--------------|----------
3Survived=yes  |  342     |  274 (80%)   |  68 (20%)
4Survived=no   |  549     |  439 (80%)   |  110 (20%)
```

এতে model balanced data দেখে শেখে এবং evaluation-ও fair হয়।

Summary

`train_test_split` দিয়ে আমরা data-কে 80% train এবং 20% test-এ ভাগ করেছি।

`random_state=42` নিশ্চিত করে যে প্রতিবার একই ভাগ হবে। `stratify=y` নিশ্চিত করে যে train এবং test উভয় set-এ survived এবং not survived-এর অনুপাত সমান থাকবে।

Outlier Handling

Train-Test Split-এর পরে আমাদের কাছে `X_train`, `X_test`, `y_train`, `y_test` ready আছে। এখন model train করার আগে একটি গুরুত্বপূর্ণ কাজ করতে হবে — **Outlier Handling**। Dataset-এ এমন কিছু value থাকতে পারে যেগুলো বাকি data থেকে অনেক বেশি বা অনেক কম — এই extreme values-কে বলা হয় **Outlier**।

Outlier কেন সমস্যা?

ধরো একটি class-এ 30 জন student-এর বয়স 18 থেকে 25-এর মধ্যে, কিন্তু একজনের বয়স 95। এই একটি value পুরো dataset-এর **mean, standard deviation** সব পরিবর্তন করে দেবে। Model এই extreme value থেকে ভুল pattern শিখবে এবং নতুন data-তে ভালো কাজ করবে না।

এই module-এ আমরা দুটি column-এ outlier handle করেছি — `Age` এবং `Fare`। কিন্তু দুটির জন্য দুটি আলাদা method ব্যবহার করা হয়েছে।

Age Column — Z-score Method

`Age` column-এ outlier **remove** করা হয়েছে — অর্থাৎ যে rows-এ extreme age আছে সেগুলো train set থেকে বাদ দেওয়া হয়েছে।

Z-score হলো একটি measure যা বলে একটি value তার mean থেকে কতটা দূরে আছে standard deviation-এর হিসেবে।

```
Z-score = (value - mean) / standard deviation
```

Z-score যদি 0 হয়, মানে value ঠিক mean-এ আছে। Z-score যদি +3 বা -3 হয়, মানে value mean থেকে 3 standard deviation দূরে — এটি একটি extreme value।

```
# Train set-এর Age-এর mean এবং std বের করা
mean_age = X_train['Age'].mean()
std_age  = X_train['Age'].std()



# প্রতিটি row-এর Z_score calculate করা
X_train['Z_score'] = (X_train['Age'] - mean_age) / std_age

# শুধু |Z_score| <= 3 রাখা, বাকি বাদ
mask = (abs(X_train['Z_score']) <= 3)
X_train = X_train[mask]
y_train = y_train[mask]
```

লক্ষ্য করো — `y_train`-কেও একই mask দিয়ে filter করা হয়েছে। কারণ `X_train` এবং `y_train`-এর row সংখ্যা সবসময় same রাখতে হবে।

Fare Column — IQR Clipping Method

`Fare` column-এ outlier **remove** করা হয়নি — বরং extreme values-কে একটি নির্দিষ্ট range-এর মধ্যে **clip** করা হয়েছে। Age-এর মতো rows বাদ না দিয়ে values-কে boundary-তে নামিয়ে আনা হয়েছে।

IQR (Interquartile Range) হলো data-এর middle 50% এর range।

```
Q1  = 25th percentile (নিচের 25% এর boundary)
Q3  = 75th percentile (উপরের 25% এর boundary)
IQR = Q3 - Q1
```

এই IQR দিয়ে আমরা একটি acceptable range তৈরি করি:

```
minimum = max(0, Q1 - 1.5 * IQR)
maximum = Q3 + 1.5 * IQR
```

এই range-এর বাইরের যেকোনো value-কে boundary-তে নামিয়ে আনা হয়:

```
fare_Q1 = X_train['Fare'].quantile(0.25)
fare_Q3 = X_train['Fare'].quantile(0.75)
IQR     = fare_Q3 - fare_Q1

minimum = max(0, fare_Q1 - 1.5 * IQR)
maximum = fare_Q3 + 1.5 * IQR

# Fare column-কে [minimum, maximum] range-এ clip করা
X_train['Fare'] = X_train['Fare'].clip(minimum, maximum)
```

দুটি Method কেন আলাদা?

```
Column  |  Method    |  কারণ
--------|------------|-----------------------------------------------------
Age     |  Remove    |  Age-এর extreme value সত্যিই impossible
Fare    |  Clipping  |  Fare বেশি হওয়া possible, তাই row বাদ নয়
```

একজন passenger-এর বয়স 200 হওয়া সম্ভব নয় — তাই সেই row বাদ দেওয়া যায়। কিন্তু কেউ অনেক বেশি দামের ticket কিনতেই পারে — তাই সেই row বাদ না দিয়ে শুধু value টি clip করা হয়।

Outlier শুধু Train Set-এ কেন?

একটি গুরুত্বপূর্ণ বিষয় লক্ষ্য করো — outlier handling শুধু `X_train`-এ করা হয়েছে, `X_test`-এ নয়।

কারণ test set হলো "unseen data" — real world-এ যে data আসবে তার প্রতিনিধি। Real world-এ outlier আসতেই পারে। Model-কে সেই পরিস্থিতি handle করতে শিখতে হবে। Train set clean রাখলে model ভালো pattern শেখে, কিন্তু test set-এ real scenario maintain করা হয়।

Summary

Outlier হলো এমন extreme value যা model-এর learning-কে ক্ষতিগ্রস্ত করে। `Age` column-এ Z-score method দিয়ে extreme rows remove করা হয়েছে এবং `Fare` column-এ IQR method দিয়ে extreme values clip করা হয়েছে। Outlier handling সবসময় শুধু train set-এ করতে হয়, test set-এ নয়।